

CS343 Operating Systems

Lecture 14

Introduction to Segmentation and Paging



John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Logical vs. Physical Address Space

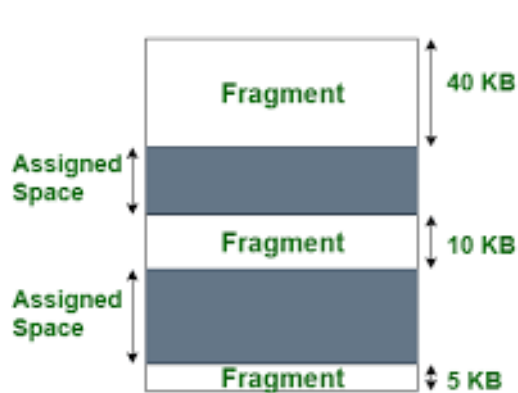
- ❖ **Logical address** – generated by the CPU; also referred to as **virtual address**
- ❖ **Physical address** – address seen by the memory unit
- ❖ **Logical address space** is the set of all logical addresses generated by a program
- ❖ **Physical address space** is the set of all physical addresses allowed for a program

Contiguous allocation and External Fragmentation

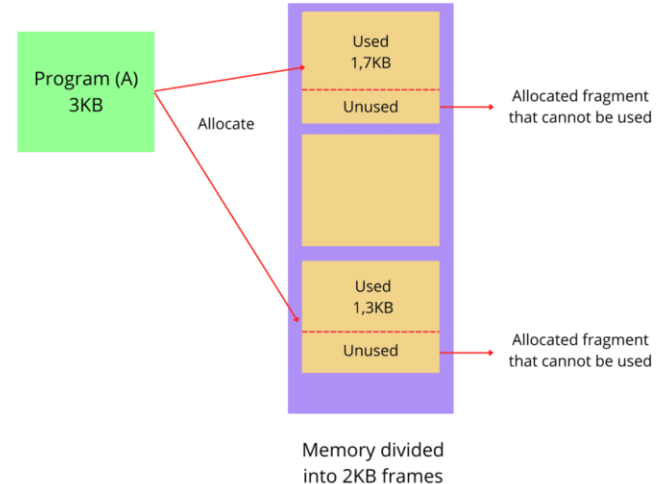
- ❖ Multiple-partition allocation to support contiguous allocation
 - ❖ **Variable-partition** sizes for efficiency as per size of process
 - ❖ **Hole** – block of available memory; holes of various size are scattered throughout memory
 - ❖ When a process arrives, it is allocated memory from a hole large enough to accommodate it

Fragmentation

- ❖ **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- ❖ **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used



Process 07
needs 50KB
memory space



Fragmentation

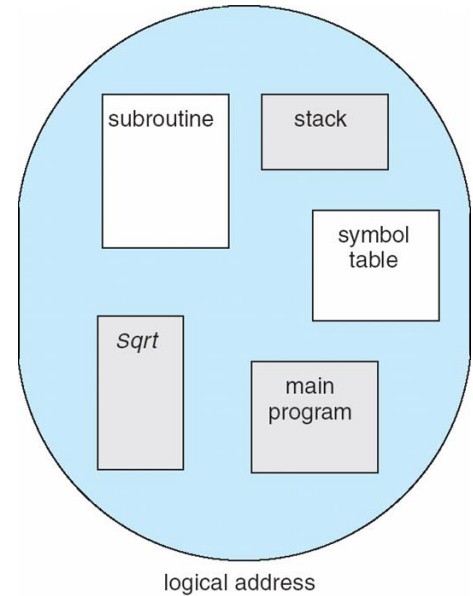
- ❖ Reduce external fragmentation by **compaction**
 - ❖ Shuffle memory contents to place all free memory together in one large block
 - ❖ Compaction is possible *only* if relocation is dynamic, and is done at execution time
 - ❖ I/O problem: Latch job in memory while it is involved in I/O

Principle of Locality

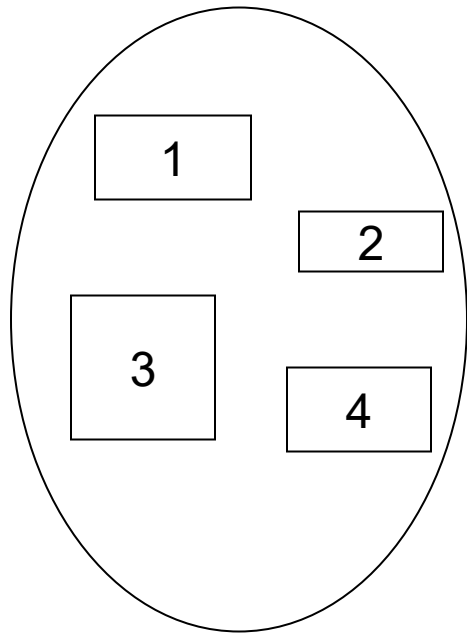
- ❖ Program and data references within a process tend to cluster around an address space. Only a few pieces of a process will be needed over a short period of time
- ❖ This help us to make intelligent guesses about which pieces will be needed in the future
- ❖ To implement virtual memory, hardware must support paging and segmentation
- ❖ OS must do the movement of pieces of process between main and secondary memory
- ❖ Physical address space of a process can be noncontiguous; - avoids external fragmentation

Segmentation

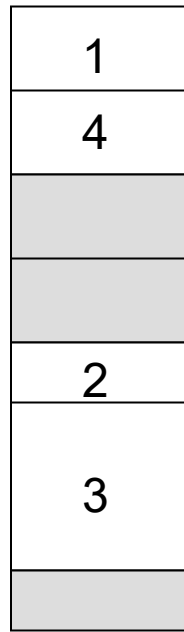
- ❖ Memory-management scheme that supports user view of memory
- ❖ A program is a collection of segments
 - ❖ A segment is a logical unit such as:
 - ❖ main program
 - ❖ procedure
 - ❖ function
 - ❖ symbol table
 - ❖ stack



Logical View of Segmentation



user space

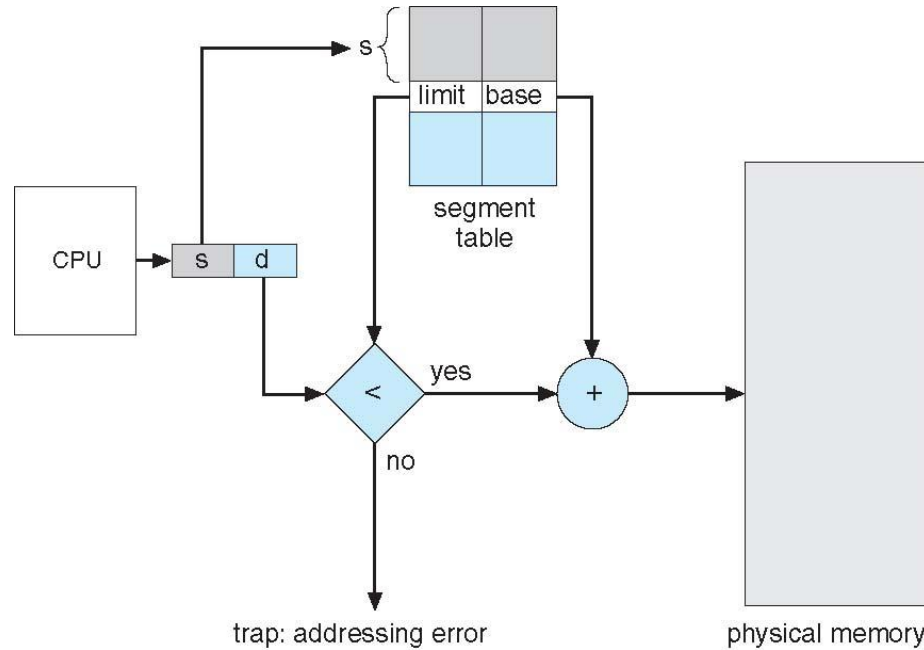


physical memory space

Segmentation Architecture

- ❖ Logical address consists of a two tuple: <segment-number, offset> ,
- ❖ **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - ❖ **base** – contains the starting physical address where the segments reside in memory
 - ❖ **limit** – specifies the length of the segment
- ❖ **Segment-table base register (STBR)** points to the segment table's location in memory
- ❖ **Segment-table length register (STLR)**

Segmentation Hardware

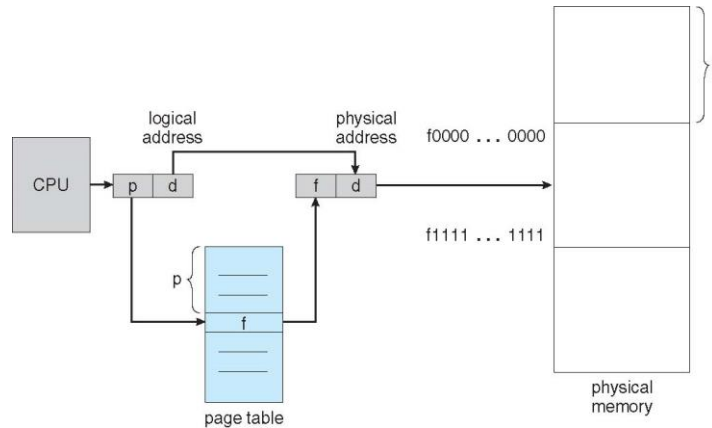


Paging

- ❖ Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
 - ❖ Avoids external fragmentation
 - ❖ Avoids problem of varying sized memory chunks
- ❖ Divide physical memory into fixed-sized blocks called **frames**
 - ❖ Size is power of 2, between 512 bytes and 16 Mbytes
- ❖ Divide logical memory into blocks of same size called **pages**
- ❖ Keep track of all free frames
- ❖ To run a program of size **N** pages, need to find **N** free frames
- ❖ Set up a **page table** to translate logical to physical addresses

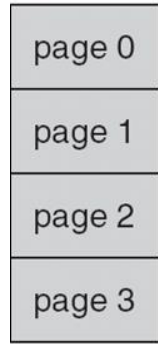
Address Translation Scheme

- ❖ Address generated by CPU is divided into:
 - ❖ **Page number** (p) – used as an index into a page table
 - ❖ **Page offset** (d) – displacement within a page



page number	page offset
p	d
$m - n$	n

Paging Model and Page Tables

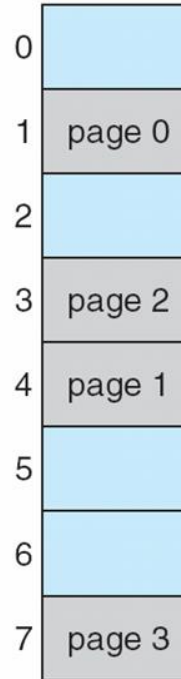


logical
memory

0	1
1	4
2	3
3	7

page table

frame
number



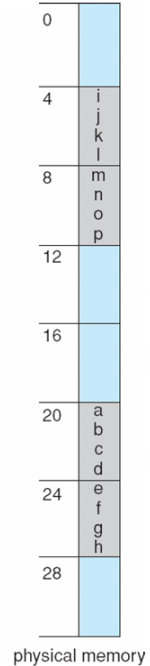
physical
memory

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

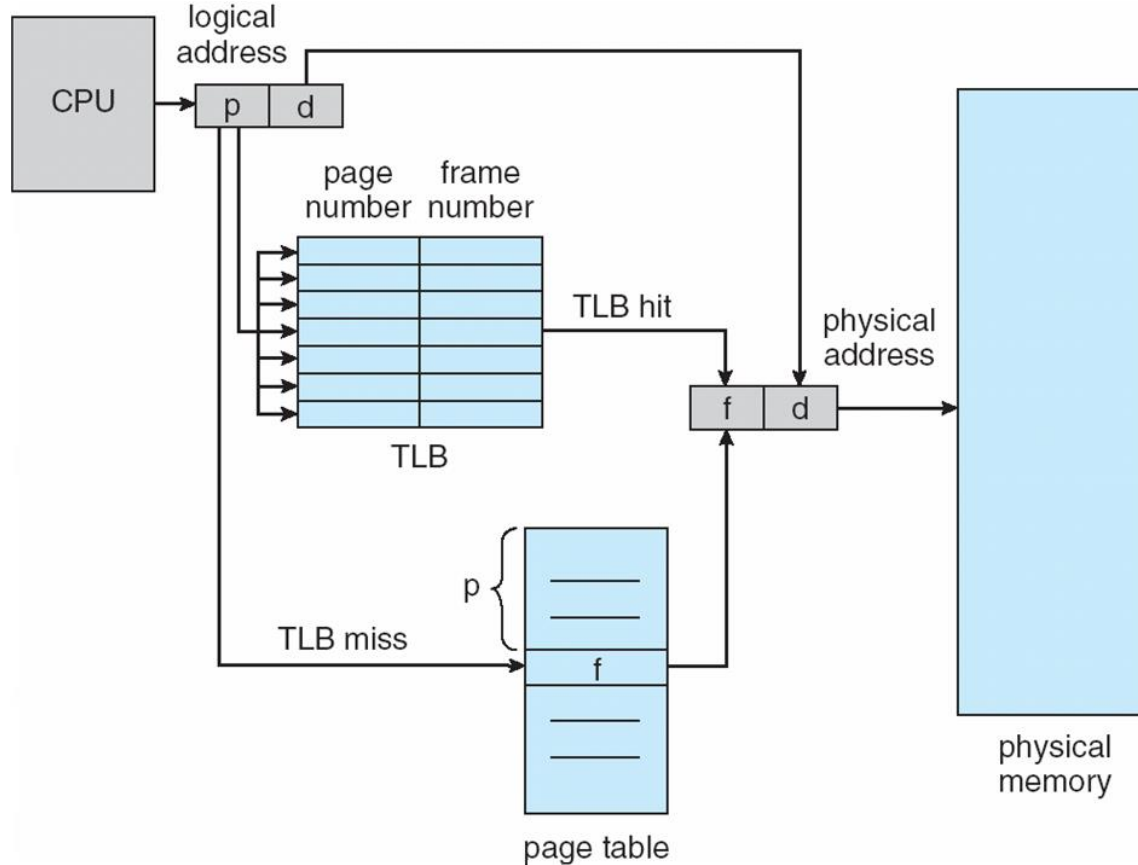
page table



Implementation of Page Table

- ❖ Page table is kept in main memory
- ❖ **Page-table base register (PTBR)** points to the page table
- ❖ **Page-table length register (PTLR)** indicates size of the page table
- ❖ In this scheme every data/instruction access requires two memory accesses : one for the page table and one for the data / instruction
- ❖ The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

Paging Hardware With TLB





johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>